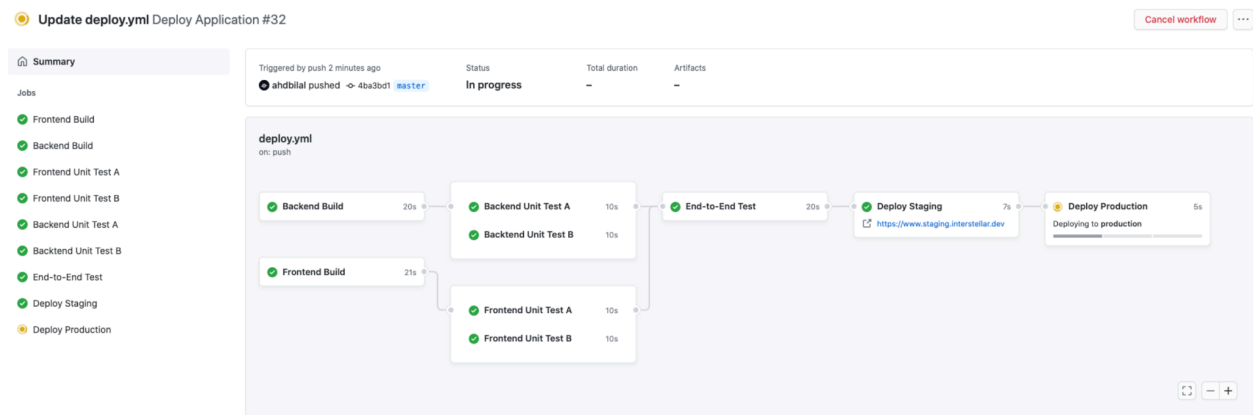


Pipelines de Validaciones y Deploy con GitHub Actions

Objetivo:

Incorporar un proceso básico de **Integración Continua** en el proyecto AIDA utilizando GitHub Actions. Vamos a configurar un workflow que, cada vez que subamos código, ejecute automáticamente los pasos esenciales para validar la calidad del proyecto: instalar dependencias, compilar la aplicación y correr tests.

Además, vamos a agregar un proceso de **deploy controlado**, que solo se ejecuta si el código pasó todas las verificaciones y cuenta con una aprobación manual. Al finalizar la clase, cada grupo contará con un workflow funcional que garantiza que no se deploye código con errores y que el proceso de despliegue sea más seguro y ordenado.



0. Encuesta

Creamos una nueva encuesta para que nos cuenten un poco como vienen experimentando la cursada. Por favor, los que todavía no lo hayan hecho, intenten completarla, que nos ayuda a mejorar!

<https://docs.google.com/forms/d/e/1FAIpQLScb1mloBNP0UhYo1YsiblintleUzShdBdNyNhAK8GiumGFoBQ/viewform?usp=dialog>

1. Introducción

Hasta ahora, los deploys de la aplicación AIDA se vienen haciendo **de forma manual**, directamente desde Render. Cada vez que subimos un cambio al repositorio, vamos a render y realizamos un deploy de la versión que queremos. Este proceso funciona, pero tiene

limitaciones. Si alguien sube una versión con errores o no compila, Render igual intenta deployarlo.

Integración Continua (CI)

La **Integración Continua (CI)** consiste en automatizar la verificación del código cada vez que alguien hace un cambio en el repositorio. Se busca ahora que un servidor ejecute una serie de pasos automáticamente con distintas tareas.

Esto permite detectar errores temprano, antes de que lleguen a producción, y garantiza que cada cambio cumpla con un estándar mínimo de calidad y sino bloquearlo

Usos comunes de Pipelines:

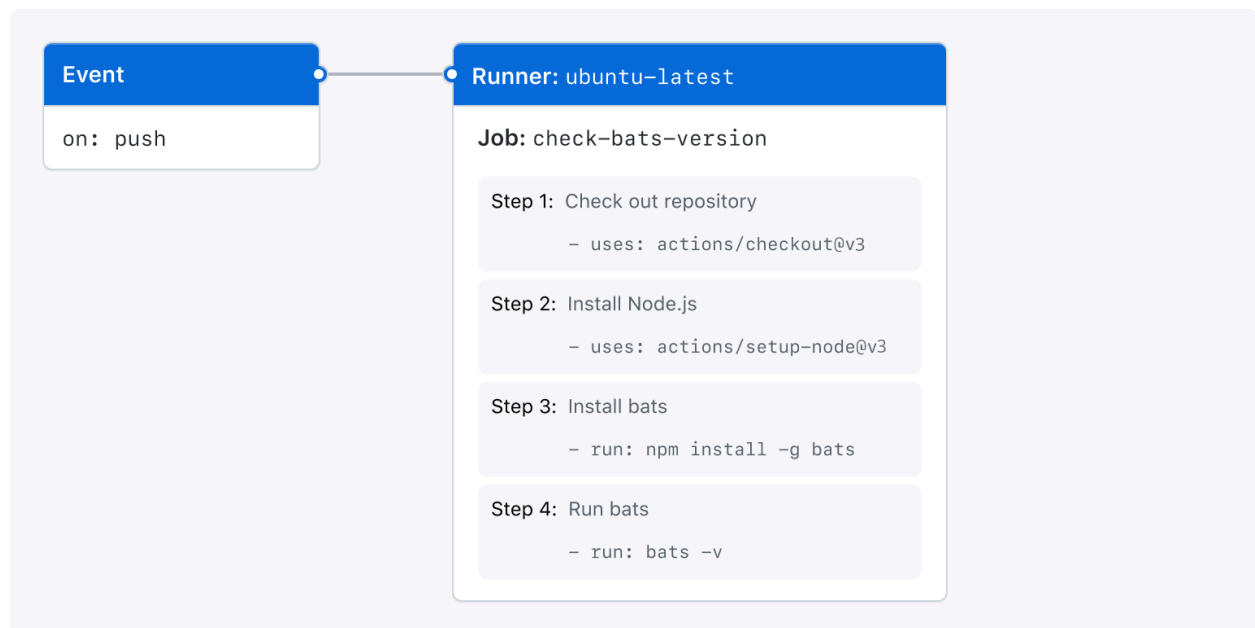
- Verificar que la aplicación compile correctamente y ejecutar tests automáticamente en cada commit.
- Validar PRs antes de permitir que se mergeen (gates de calidad)
- Publicar versiones en producción o staging
- Ejecutar tareas programadas (backups, limpieza, reportes)

GitHub Actions

Es una herramienta integrada en GitHub que permite automatizar tareas cada vez que ocurre un evento en el repositorio, como un push o un pull request. Se crea un *runner* (una máquina virtual) que sigue las instrucciones definidas en un archivo de configuración y ejecuta automáticamente los pasos que le digamos

La clave es que estas automatizaciones se definen mediante **workflows**, que son archivos escritos en formato declarativo dentro de la carpeta `.github/workflows/`.

Dentro de cada workflow podemos definir **jobs** (un trabajo a realizar) y **steps** (pasos concretos a ejecutar dentro de cada job). Esto nos permite crear pipelines que corren siempre de forma consistente y reproducible, sin depender del entorno de cada persona y sin intervención manual.



2. Workflows

YAMLs

Un archivo **YAML** (extensión `.yaml` o `.yml`) es un archivo de **configuración** escrito en texto plano. Su objetivo es describir información de manera **simple, ordenada y fácil de leer por humanos**. En YAML todo se organiza usando:

- **clave: valor**

- listas
- bloques

```
proyecto: AIDA
version: 1.0

equipo:
  - "Juan"
  - "Martina"
  - "Sofía"

configuracion:
  lenguaje: "Node.js"
  base_datos: "Supabase"
```

Este ejemplo muestra:

- **clave: valor** → **proyecto: AIDA**
- **bloques** → **configuracion**
- **lista** → **equipo: - Juan - Martina - Sofía**

La indentación es lo más importante: en YAML **no se usan llaves ni corchetes**, sino que la estructura se define moviendo las líneas hacia la derecha

YAML se usa muchísimo en el mundo real: en GitHub Actions, configuraciones de servidores y muchas herramientas modernas de desarrollo.

Mi primer Workflow

Debajo tenemos un workflow que corre cada vez que hacemos un push a cualquier rama. Este ejecuta un único job que primero baja el código del repositorio y luego corre dos comandos simples: mostrar un saludo por consola y listar los archivos del proyecto.

```
name: MiPrimerWorkflow

on:
  push:
    branches:
      - '*' # cualquier branch

jobs:
  saludo:
    runs-on: ubuntu-latest

    steps:
      - name: Descargar el repositorio
        uses: actions/checkout@v4

      - name: Saludar por consola
        run: echo "Hola! Este es mi primer pipeline"

      - name: Mostrar archivos del proyecto
        run: ls -la
```

Ahora intenten ustedes crear su primer workflow que

1. Baje el repositorio
2. Compile la aplicacion
3. Corra los tests

Para eso creen el archivo `.github/workflows/miPrimerWorkflow.yml`

Importante!!!!: Los Github Actions solo pueden usarse de forma gratuita en repositorios públicos. Van a tener que actualizarlo sino no va a andar

 **sbocaccio**

Cambio Branch a la que Corre

025100d · now

 73 Commits

 .github/workflows

Cambio Branch a la que Corre

now

  ari-dc-uba-ar / aida

Q Type to search

[Code](#) [Issues](#) [Pull requests](#) 2 [Actions](#) [Projects](#) [Security](#) [Insights](#) [Settings](#)

[← MiPrimerWorkflow](#)

 **Cambio Branch a la que Corre #22**

Re-run all jobs

...

 Summary

Jobs

 **saludo**

Run details

 Usage

 Workflow file

saludo

succeeded now in 3s

Q Search logs

>  Set up job

1s

>  Descargar el repositorio

0s

>  Saludar por consola

0s

>  Mostrar archivos del proyecto

0s

>  Post Descargar el repositorio

1s

>  Complete job

0s

Complejizando los Workflows

El siguiente workflow se ejecuta tanto cuando hay un *push* como cuando hay un *pull_request*. También se agrega un job2 que se ejecuta solamente en la branch *main* y luego de job1 haya finalizado ya que depende de é

```
name: MiSegundoWorkflow

on:
  push:
    branches:
      - '*' # cualquier branch
  pull_request:
    branches:
      - '*' # opcional, por si querés que también funcione en PR

jobs:
  job1:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Step del Job 1
        run: echo "Este es el Job 1"

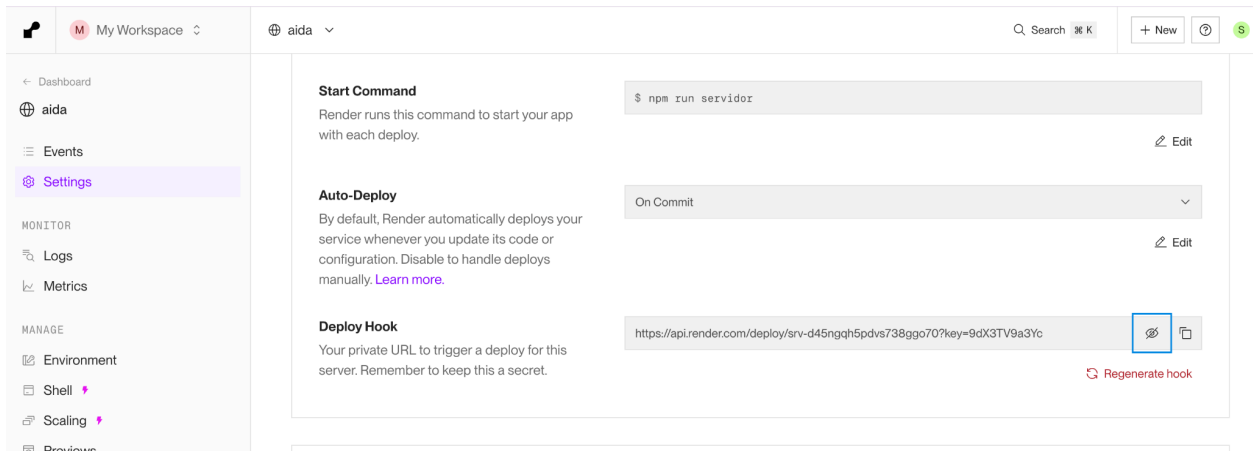
  job2:
    needs: job1
    if: github.ref == 'refs/heads/main' # solo corre en main
    runs-on: ubuntu-latest
    steps:
      - name: Step del Job 2
        run: echo "Este es el Job 2 (solo main)"
```

Actualicen el Workflow que tienen actualmente para que ahora:

- El Job que baja el repo, compila y corre los tests se haga tanto en *push* como *pull_request*
- Agreguen un segundo job que tan solo se ejecute cuando se pushee a main. Por ahora con que logee un mensaje alcanza.

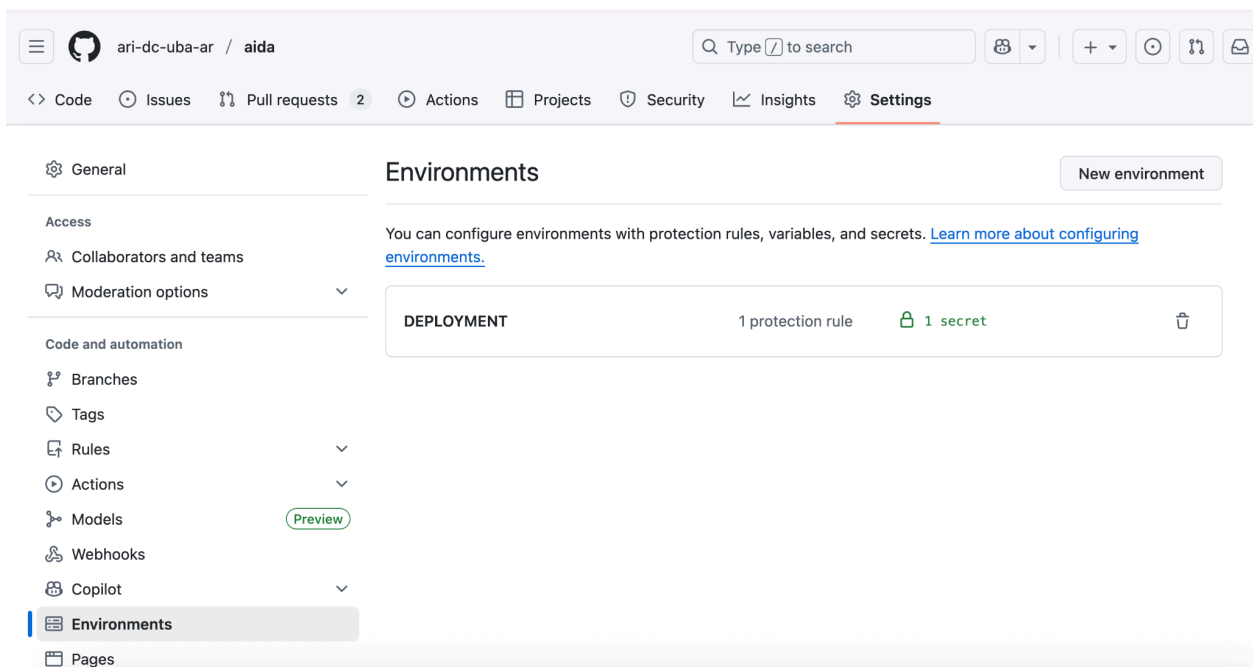
Preparar nuestro Repo de Github para permitir Deploys automaticos

Cuando nosotros realizamos una request de tipo **GET** a la URL de *Deploy Hook* Render va a deployar nuestra aplicación de forma automática



Guárdense ese valor que vamos a usar luego y recuerden que es un valor privado que no debe compartirse nunca.

Ahora vayan a su repositorio de Github, luego a *Settings >> Environment*. Creen un nuevo ambiente que sea DEPLOYMENT



Luego, van a agregarse como nuevos *Reviewers* de ese ambiente. Esto hace que ustedes tengan permiso de autorizar acciones que requieren acciones manuales en los Jobs que utilizan ese ambiente.

Deployment protection rules

Configure reviewers, timers, and custom rules that must pass before deployments to this environment can proceed.

☒ **Required reviewers**

Specify people or teams that may approve workflow runs when they access this environment.

Add up to 3 more reviewers

Search for people or teams...

 sebasbocaccio

×

 sbocaccio

×

 emilioplatzer

×

☐ **Prevent self-review**

Require a different approver than the user who triggered the workflow run.

☐ **Wait timer**

Set an amount of time to wait before allowing deployments to proceed.

Enable custom rules with GitHub Apps Preview

[Learn about existing apps](#) or [create your own protection rules](#) so you can deploy with confidence.

☒ **Allow administrators to bypass configured protection rules**

Save protection rules

También deben agregar un nuevo *secret* con el valor que obtuvieron anteriormente de Render.

Environment secrets

Add environment secret

Secrets are encrypted environment variables. They are accessible only by GitHub Actions in the context of this environment by using the [secret context](#).

Name 	Last updated
 URL_DEPLOYMENT	2 hours ago  

Deploy a Render cuando el Workflow finaliza de forma exitosa

Muy bien! Ahora nuestro objetivo es extender el Job2 anterior para que empiece a deployar en Render. Para eso actualizamos el workflow ambiente en el que corre y el step a ejecutar para llamar al hook.

```
job2:
  needs: job1
  if: |
    github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest

  environment:
    name: DEPLOYMENT

  steps:
    - name: Disparar deploy (GET)
      run: |
        curl "${{ secrets.URL_DEPLOYMENT }}"
```

Cuando un job de GitHub Actions declara un **environment**, GitHub aplica las reglas de protección configuradas para ese entorno. Si en ese environment activamos “**Require reviewers**”, GitHub entiende que cualquier acción asociada a ese entorno es crítica (como un deploy) y, por seguridad, **detiene el workflow antes de ejecutar el job**.

Por eso aparece un paso manual: GitHub exige que una persona revise y apruebe la ejecución antes de continuar. El YAML solo indica a qué environment pertenece el job; es la configuración del environment (no el workflow) la que obliga a esa aprobación manual.

Luego, la URL a la que dispara el deploy no se escribe directamente en el pipeline por motivos de seguridad. En su lugar, se guarda como un **secret URL_DEPLOYMENT**. Los secrets funcionan como variables de ambiente protegidas: no se ven en el código, no se imprimen en los logs y GitHub solo los expone al workflow en tiempo de ejecución

Una vez que tengan los cambios en main deberían ver algo de este estilo al tener un nuevo commit:

